

```

1 # =====
2 # SECTION 1: CLEAN ENVIRONMENT SETUP & DATA LOADING
3 # =====
4
5 # 1. Core Data Manipulation & Math Libraries
6 import numpy as np # For fast matrix and numerical operations
7 import pandas as pd # For handling structured DataFrames
8
9 # 2. Data Visualization Libraries
10 import matplotlib.pyplot as plt # Core plotting engine
11 import missingno as msno # To visualize missing data patterns
12 import plotly.express as px # For interactive, dynamic charts
13 import seaborn as sns # Advanced statistical plots
14
15 # 3. Machine Learning Models & Metrics
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.linear_model import LogisticRegression
18 import xgboost as xgb
19 from sklearn.metrics import hamming_loss, roc_auc_score
20 from sklearn.model_selection import train_test_split
21 from sklearn.preprocessing import StandardScaler
22
23 print("🧰 Core data science and ML libraries loaded successfully!")
24
25 # =====
26 # DATA LOADING FROM LOCAL HARD DRIVE
27 # =====
28
29 # =====
30 # ONLY DATASET UPLOADING & LOADING PART
31 # =====
32 import os
33 import zipfile
34 import pandas as pd
35
36 # STEP 1: Define the path to your uploaded zip file.
37 # Since you used the "Copy path" option in Colab, paste it exactly here.
38 zip_path = "/archive.zip"
39
40 # STEP 2: Extract the contents of the zip file into Colab
41 print("📦 Unzipping your file... please wait...")
42 with zipfile.ZipFile(zip_path, "r") as zip_ref:
43     zip_ref.extractall("/content/")
44 print("📁 Unzipping complete!")
45
46 # STEP 3: Automatically find the newly extracted CSV file name
47 # This looks inside your folder and catches the CSV file automatically so you
48 # don't have to guess its name.
49 extracted_files = os.listdir("/content/")
50 csv_file = [file for file in extracted_files if file.endswith(".csv")][0]
51 print(f"📄 Found extracted CSV file: {csv_file}")
52
53 # STEP 4: Load the CSV file into a Pandas DataFrame called 'data'
54 print("⌚ Loading CSV data into memory...")
55 data = pd.read_csv(f"/content/{csv_file}")
56
57 print("\n🚀 SUCCESS! Your dataset is fully loaded and ready to use.")
58 print(f"Dataset Shape: {data.shape[0]} rows, {data.shape[1]} columns")

```

```

🧰 Core data science and ML libraries loaded successfully!
📦 Unzipping your file... please wait...
📁 Unzipping complete!
📄 Found extracted CSV file: PS_20174392719_1491204439457_log.csv
⌚ Loading CSV data into memory...

```

```

🚀 SUCCESS! Your dataset is fully loaded and ready to use.
Dataset Shape: 6362620 rows, 11 columns

```

```

1 data
2

```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDe
0	1	PAYMENT	9839.64	C1231006815	170136.00	160296.36	M1979787155	0.00	0
1	1	PAYMENT	1864.28	C1666544295	21249.00	19384.72	M2044282225	0.00	0
2	1	TRANSFER	181.00	C1305486145	181.00	0.00	C553264065	0.00	0
3	1	CASH_OUT	181.00	C840083671	181.00	0.00	C38997010	21182.00	0
4	1	PAYMENT	11668.14	C2048537720	41554.00	29885.86	M1230701703	0.00	0
...	...	...	...	...	...	...	...	...	...
6362615	743	CASH_OUT	339682.13	C786484425	339682.13	0.00	C776919290	0.00	339682
6362616	743	TRANSFER	6311409.28	C1529008245	6311409.28	0.00	C1881841831	0.00	0
6362617	743	CASH_OUT	6311409.28	C1162922333	6311409.28	0.00	C1365125890	68488.84	6379898
6362618	743	TRANSFER	850002.52	C1685995037	850002.52	0.00	C2080388513	0.00	0
6362619	743	CASH_OUT	850002.52	C1280323807	850002.52	0.00	C873221189	6510099.11	7360101

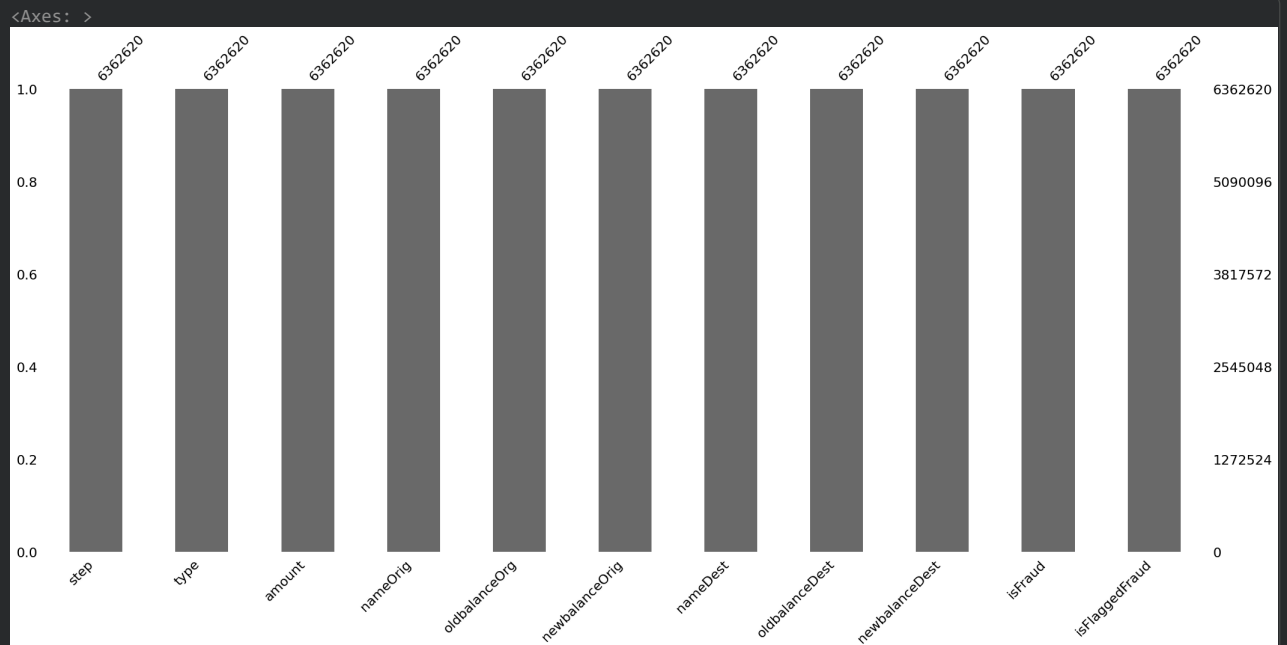
6362620 rows × 11 columns

1

```
1 data.info()
2 print("-----SHAPE-----")
3 data.shape
4
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
#   Column          Dtype
---  ---
0   step            int64
1   type            object
2   amount          float64
3   nameOrig        object
4   oldbalanceOrg   float64
5   newbalanceOrig  float64
6   nameDest        object
7   oldbalanceDest  float64
8   newbalanceDest  float64
9   isFraud         int64
10  isFlaggedFraud  int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
-----SHAPE-----
(6362620, 11)
```

```
1 msno.bar(data)
2
```



```
1 duplicate_count = len(data[data.duplicated()])
2 duplicate_count
```

0

```
1 new_column_names = {
2     'nameOrig': 'origin',
3     'oldbalanceOrig': 'sender_old_bal',
4     'newbalanceOrig': 'sender_new_bal',
5     'nameDest': 'dest',
6     'oldbalanceDest': 'dest_old_bal',
7     'newbalanceDest': 'dest_new_bal'
8 }
9
10 data.rename(columns=new_column_names, inplace=True)
```

1 data

	step	type	amount	origin	sender_old_bal	sender_new_bal	dest	dest_old_bal	dest_new_bal
0	1	PAYMENT	9839.64	C1231006815	170136.00	160296.36	M1979787155	0.00	0.00
1	1	PAYMENT	1864.28	C1666544295	21249.00	19384.72	M2044282225	0.00	0.00
2	1	TRANSFER	181.00	C1305486145	181.00	0.00	C553264065	0.00	0.00
3	1	CASH_OUT	181.00	C840083671	181.00	0.00	C38997010	21182.00	0.00
4	1	PAYMENT	11668.14	C2048537720	41554.00	29885.86	M1230701703	0.00	0.00
...	...	...	...	...	...	...	...	...	...
6362615	743	CASH_OUT	339682.13	C786484425	339682.13	0.00	C776919290	0.00	339682.13
6362616	743	TRANSFER	6311409.28	C1529008245	6311409.28	0.00	C1881841831	0.00	0.00
6362617	743	CASH_OUT	6311409.28	C1162922333	6311409.28	0.00	C1365125890	68488.84	6379898.11
6362618	743	TRANSFER	850002.52	C1685995037	850002.52	0.00	C2080388513	0.00	0.00
6362619	743	CASH_OUT	850002.52	C1280323807	850002.52	0.00	C873221189	6510099.11	7360101.63

6362620 rows x 11 columns

```
1 columns_to_drop = ['step', 'isFlaggedFraud']
2 data.drop(columns=columns_to_drop, inplace=True)
```

1 data

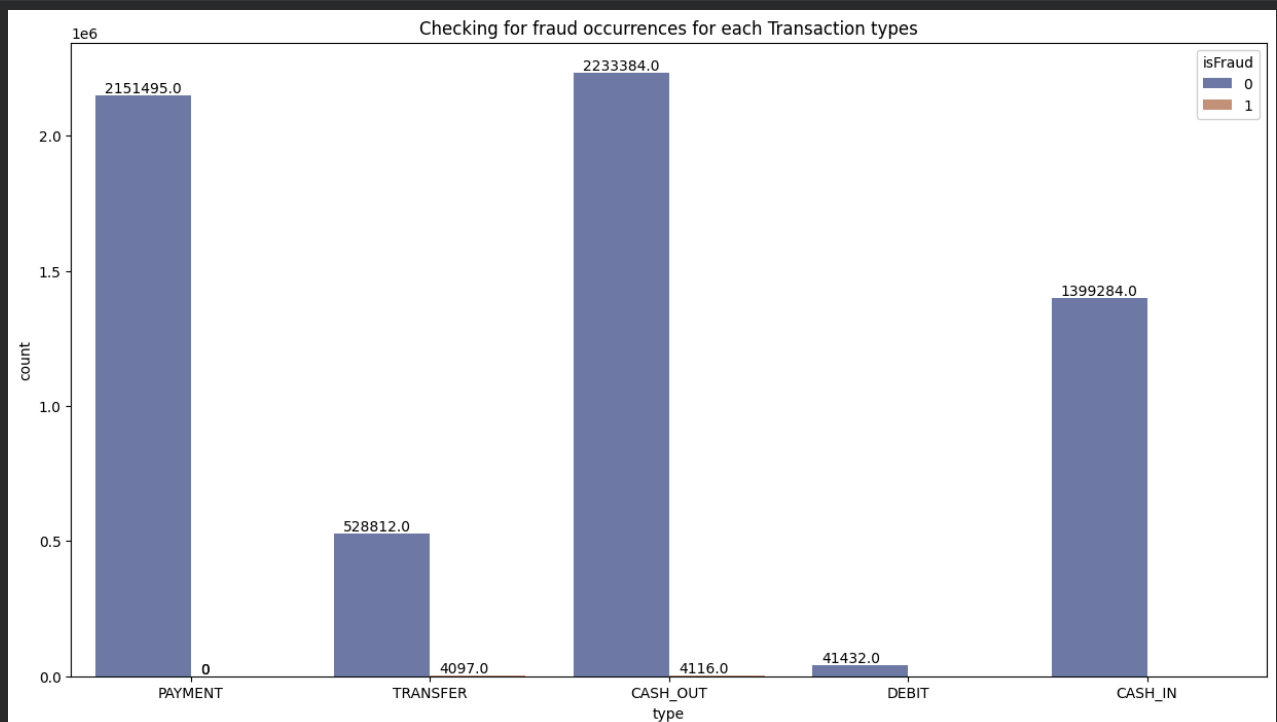
	type	amount	origin	sender_old_bal	sender_new_bal	dest	dest_old_bal	dest_new_bal	isFraud
0	PAYMENT	9839.64	C1231006815	170136.00	160296.36	M1979787155	0.00	0.00	
1	PAYMENT	1864.28	C1666544295	21249.00	19384.72	M2044282225	0.00	0.00	
2	TRANSFER	181.00	C1305486145	181.00	0.00	C553264065	0.00	0.00	
3	CASH_OUT	181.00	C840083671	181.00	0.00	C38997010	21182.00	0.00	
4	PAYMENT	11668.14	C2048537720	41554.00	29885.86	M1230701703	0.00	0.00	
...	...	...	...	...	...	...	...	...	...
6362615	CASH_OUT	339682.13	C786484425	339682.13	0.00	C776919290	0.00	339682.13	
6362616	TRANSFER	6311409.28	C1529008245	6311409.28	0.00	C1881841831	0.00	0.00	
6362617	CASH_OUT	6311409.28	C1162922333	6311409.28	0.00	C1365125890	68488.84	6379898.11	
6362618	TRANSFER	850002.52	C1685995037	850002.52	0.00	C2080388513	0.00	0.00	
6362619	CASH_OUT	850002.52	C1280323807	850002.52	0.00	C873221189	6510099.11	7360101.63	

6362620 rows × 9 columns

```
1 def plot_fraud_by_transaction_type(data):
2     plt.figure(figsize=(15, 8))
3     g = sns.countplot(data=data, x='type', hue='isFraud', palette='dark', alpha=0.6)
4     plt.title("Checking for fraud occurrences for each Transaction types")
5
6     # Finding counts and annotating
7     annotate_counts(g)
```

```
1 def annotate_counts(ax):
2     for p in ax.patches:
3         ax.annotate(p.get_height(), (p.get_x() + 0.04, p.get_height() + 10000))
```

```
1 plot_fraud_by_transaction_type(data)
2 plt.show()
3
```



```
1 transfer_fraud = data[(data['type'] == 'TRANSFER') & (data['isFraud'] == 1)]
2 transfer_fraud_counts = data[(data['type'] == 'TRANSFER') & (data['isFraud'] == 1)]['origin'].value_counts()
```

```
1 transfer_fraud_counts
```

	count
origin	
C1685995037	1
C1565612524	1
C992223106	1
C1945606464	1
C88849251	1
...	...
C669700766	1
C1364127192	1
C1334405552	1
C137533655	1
C1420196421	1

4097 rows × 1 columns

dtype: int64

```
1 cashOut_fraud = data[(data['type']=='CASH_OUT') & (data['isFraud']==1)]
2 cashOut_fraud_count = data[(data['type']=='CASH_OUT') & (data['isFraud']==1)]['origin'].value_counts()
3
4 cashOut_fraud_count
```

	count
origin	
C1280323807	1
C840083671	1
C2101527076	1
C1118430673	1
C1597957921	1
...	...
C1499825229	1
C13692003	1
C1275464847	1
C1635772897	1
C467632528	1

4116 rows × 1 columns

dtype: int64

```
1 transfer_fraud['dest'].value_counts()
```

```

count
dest
C2080388513    1
C435734396     1
C1366804249    1
C625944676     1
C1939028448    1
...
C1384210339    1
C1136419747    1
C431687661     1
C1848415041    1
C972765878     1
4097 rows × 1 columns

```

dtype: int64

```
1 cashOut_fraud['dest'].value_counts()
```

```

count
dest
C1650668671    2
C644163395     2
C1013511446    2
C2129197098    2
C185805228     2
...
C1955464150    1
C1104137869    1
C377165949     1
C576226769     1
C742045843     1
4091 rows × 1 columns

```

dtype: int64

```

1 is_dest_in_cashout_origin = transfer_fraud['dest'].isin(cashOut_fraud['origin']).any()
2 trans_cashout = data[data['type'] == 'TRANSFER']
3 is_dest_in_trans_cashout_origin = transfer_fraud['dest'].isin(trans_cashout['origin']).any()
4 count_dest_in_trans_cashout_origin = transfer_fraud['dest'].isin(trans_cashout['origin']).sum()
5 print(f"Is destination in cashOut origin? {is_dest_in_cashout_origin}")
6 print(f"Is destination in trans_cashout origin? {is_dest_in_trans_cashout_origin}")
7 print(f"Count of destinations in trans_cashout origin: {transfer_fraud.dest.isin(trans_cashout.origin).count()}")

```

```

Is destination in cashOut origin? False
Is destination in trans_cashout origin? True
Count of destinations in trans_cashout origin: 4097

```

Feature Engineering Creating new column `client_type` having 4 categories and dropping 'origin' and 'dest'

CC - Customer to Customer
CM - Customer to Merchant
MC - Merchant to Customer
MM - Merchant to Merchant

```
1 print(data.columns)
```

```

Index(['type', 'amount', 'sender_old_bal', 'sender_new_bal', 'dest_old_bal',
       'dest_new_bal', 'isFraud'],
      dtype='object')

```

```

1 # 1. Filter the dataset to keep ONLY TRANSFER and CASH_OUT transactions
2 cleaned_df = data[data["type"].isin(["TRANSFER", "CASH_OUT"])].copy()
3
4 # 2. Calculate balance error for origin (sender) account
5 # In a perfect transaction: old_balance - amount = new_balance.
6 cleaned_df["errorBalanceOrig"] = (
7     cleaned_df["sender_new_bal"]
8     + cleaned_df["amount"]
9     - cleaned_df["sender_old_bal"]
10 )
11
12 # 3. Calculate balance error for destination (receiver) account
13 # In a perfect transaction: old_balance + amount = new_balance.
14 cleaned_df["errorBalanceDest"] = (
15     cleaned_df["dest_old_bal"] + cleaned_df["amount"] - cleaned_df["dest_new_bal"]
16 )
17
18 # 4. One-hot encode the 'type' column (converts TRANSFER/CASH_OUT into 1 and 0)
19 cleaned_df = pd.get_dummies(cleaned_df, columns=["type"], drop_first=True)
20
21 print("Cell 2 Success! Cleaned and engineered features successfully.")
22 print(f"Final dataset shape ready for ML: {cleaned_df.shape}")
23 cleaned_df.head(3)

```

Cell 2 Success! Cleaned and engineered features successfully.
Final dataset shape ready for ML: (2770409, 9)

	amount	sender_old_bal	sender_new_bal	dest_old_bal	dest_new_bal	isFraud	errorBalanceOrig	errorBalanceDest	type_
2	181.00	181.0	0.0	0.0	0.00	1	0.00	181.0	
3	181.00	181.0	0.0	21182.0	0.00	1	0.00	21363.0	
15	229133.94	15325.0	0.0	5083.0	51513.44	0	213808.94	182703.5	

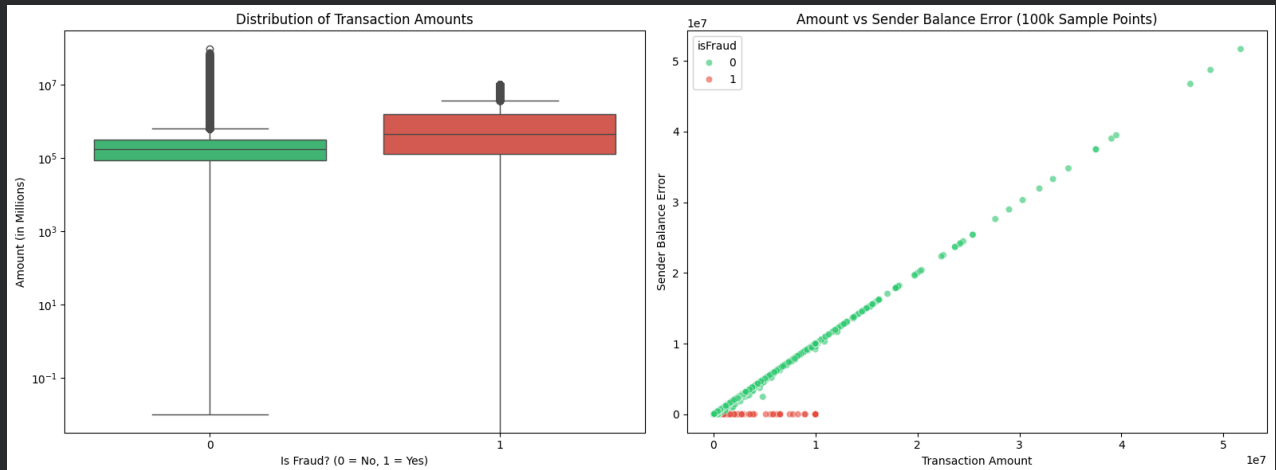
```

1 # =====
2 # DATA VISUALIZATION SECTION
3 # =====
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 # Create a figure canvas for two distinct plots side-by-side
8 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
9
10 # Plot 1: Boxplot of Transaction Amounts (Fraud vs. Legitimate)
11 # This shows us if fraud cases involve significantly larger sums of money
12 sns.boxplot(
13     ax=axes[0],
14     x="isFraud",
15     y="amount",
16     data=cleaned_df,
17     palette=["#2ecc71", "#e74c3c"],
18 )
19 axes[0].set_title("Distribution of Transaction Amounts")
20 axes[0].set_xlabel("Is Fraud? (0 = No, 1 = Yes)")
21 axes[0].set_ylabel("Amount (in Millions)")
22 axes[0].set_yscale("log") # Using log scale to handle extreme outliers easily
23
24 # Plot 2: Scatter Plot of Error Balance vs. Transaction Amount
25 # This visualizes how our custom engineered feature instantly separates fraud from clean data
26 sns.scatterplot(
27     ax=axes[1],
28     x="amount",
29     y="errorBalanceOrig",
30     hue="isFraud",
31     data=cleaned_df.sample(
32         100000, random_state=42
33     ), # Sampling 100k points so Colab doesn't freeze
34     palette=["#2ecc71", "#e74c3c"],
35     alpha=0.6,
36 )
37 axes[1].set_title("Amount vs Sender Balance Error (100k Sample Points)")
38 axes[1].set_xlabel("Transaction Amount")
39 axes[1].set_ylabel("Sender Balance Error")
40
41 plt.tight_layout()
42 plt.show()

```

/tmp/ipykernel_11593/2364155334.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and



```
1 # =====
2 # SECTION 5: SPLITTING THE DATA
3 # =====
4
5 # 1. Separate our features (X) from our target label (y)
6 # 'isFraud' is what we want to predict, so it goes to y. Everything else goes to X.
7 X = cleaned_df.drop(columns=["isFraud"])
8 y = cleaned_df["isFraud"]
9
10 # 2. Split the data into 80% Training and 20% Testing
11 # We use 'stratify=y' because fraud is highly imbalanced. This ensures both the train
12 # and test sets get the exact same percentage of fraud cases.
13 X_train, X_test, y_train, y_test = train_test_split(
14     X, y, test_size=0.20, random_state=42, stratify=y
15 )
16
17 print("Cell 5 Success! Data split completed safely.")
18 print(f"Training Features Shape: {X_train.shape}")
19 print(f"Testing Features Shape: {X_test.shape}")
20 print(f"Fraud cases in Training Set: {sum(y_train)}")
21 print(f"Fraud cases in Testing Set: {sum(y_test)}")
```

```
Cell 5 Success! Data split completed safely.
Training Features Shape: (2216327, 8)
Testing Features Shape: (554082, 8)
Fraud cases in Training Set: 6570
Fraud cases in Testing Set: 1643
```

```
1 # =====
2 # SECTION 6: FEATURE SCALING
3 # =====
4
5 # 1. Initialize the StandardScaler
6 scaler = StandardScaler()
7
8 # 2. Fit and transform the training data
9 # 'Fit' calculates the mean and variance of X_train.
10 # 'Transform' applies the mathematical formula to scale the data.
11 X_train_scaled = scaler.fit_transform(X_train)
12
13 # 3. Transform the testing data
14 # CRITICAL: We ONLY transform X_test. We do NOT fit it.
15 X_test_scaled = scaler.transform(X_test)
16
```


Cell 6 Success! Feature scaling applied perfectly.
Scaled Training sample (First row):
[-0.27663443 -0.19201226 -0.10914006 -0.23001149 -0.26646462 -0.24451181
0.04806802 -0.48791196]

```
1 # =====
2 # SECTION 7: LOGISTIC REGRESSION BASELINE
3 # =====
4
5 # 1. Initialize the model
6 # We use class_weight='balanced' to handle the heavy class imbalance automatically
7 log_reg = LogisticRegression(class_weight="balanced", random_state=42, max_iter=500)
8
9 # 2. Train the model on our scaled training data
10 print("⌚ Training Logistic Regression Baseline model... please wait...")
11 log_reg.fit(X_train_scaled, y_train)
12 print("🎉 Model training complete!")
13
14 # 3. Make predictions on the scaled testing data
15 y_pred = log_reg.predict(X_test_scaled)
16 y_pred_proba = log_reg.predict_proba(X_test_scaled)[
17     :, 1
18 ] # Probabilities needed for ROC-AUC
19
20 # 4. Evaluate using our specific fraud metrics
21 auc_score = roc_auc_score(y_test, y_pred_proba)
22 h_loss = hamming_loss(y_test, y_pred)
23
24 print("\n--- BASELINE EVALUATION METRICS ---")
25 print(f"ROC-AUC Score: {auc_score:.4f}")
26 print(f"Hamming Loss: {h_loss:.6f}")
```

⌚ Training Logistic Regression Baseline model... please wait...
🎉 Model training complete!

--- BASELINE EVALUATION METRICS ---
ROC-AUC Score: 0.9761
Hamming Loss: 0.065099

```
1 # =====
2 # SECTION 7.1: DETAILED CONFUSION MATRIX & CLASSIFICATION REPORT
3 # =====
4 from sklearn.metrics import classification_report, confusion_matrix
5
6 # 1. Generate the Confusion Matrix
7 # This counts True Negatives, False Positives, False Negatives, and True Positives
8 cm = confusion_matrix(y_test, y_pred)
9
10 print("--- CONFUSION MATRIX BREAKDOWN ---")
11 print(f"True Legitimate Caught (True Negatives): {cm[0][0]}")
12 print(f"False Alarms Raised (False Positives): {cm[0][1]}")
13 print(f"Missed Fraud Cases (False Negatives): {cm[1][0]}")
14 print(f"True Fraud Caught (True Positives): {cm[1][1]}")
15
16 # 2. Print the Precision, Recall, and F1-Score
17 print("\n--- DETAILED CLASSIFICATION REPORT ---")
18 print(classification_report(y_test, y_pred, target_names=["Legit", "Fraud"]))
```

--- CONFUSION MATRIX BREAKDOWN ---

True Legitimate Caught (True Negatives):	516567
False Alarms Raised (False Positives):	35872
Missed Fraud Cases (False Negatives):	198
True Fraud Caught (True Positives):	1445

--- DETAILED CLASSIFICATION REPORT ---

	precision	recall	f1-score	support
Legit	1.00	0.94	0.97	552439
Fraud	0.04	0.88	0.07	1643
accuracy			0.93	554082
macro avg	0.52	0.91	0.52	554082
weighted avg	1.00	0.93	0.96	554082

```
1 # =====
2 # SECTION 8: RANDOM FOREST MODEL
3 # =====
4 from sklearn.ensemble import RandomForestClassifier
```

```

5
6 # 1. Initialize the Random Forest
7 # We use n_estimators=50 to train faster in Colab, and class_weight='balanced' for the imbalance.
8 rf_model = RandomForestClassifier(
9     n_estimators=50, class_weight="balanced", random_state=42, n_jobs=-1
10 )
11
12 print("⌚ Training Random Forest (50 Trees)... This might take 1-2 minutes...")
13 rf_model.fit(X_train_scaled, y_train)
14 print("🎉 Random Forest training complete!")
15
16 # 2. Make predictions
17 y_pred_rf = rf_model.predict(X_test_scaled)
18 y_pred_proba_rf = rf_model.predict_proba(X_test_scaled)[: , 1]
19
20 # 3. Print the new Confusion Matrix and Classification Report
21 print("\n--- RANDOM FOREST CONFUSION MATRIX ---")
22 print(confusion_matrix(y_test, y_pred_rf))
23
24 print("\n--- RANDOM FOREST CLASSIFICATION REPORT ---")
25 print(classification_report(y_test, y_pred_rf, target_names=["Legit", "Fraud"]))

```

⌚ Training Random Forest (50 Trees)... This might take 1-2 minutes...
 🎉 Random Forest training complete!

--- RANDOM FOREST CONFUSION MATRIX ---

```
[[552439    0]
 [    6  1637]]
```

--- RANDOM FOREST CLASSIFICATION REPORT ---

	precision	recall	f1-score	support
Legit	1.00	1.00	1.00	552439
Fraud	1.00	1.00	1.00	1643
accuracy			1.00	554082
macro avg	1.00	1.00	1.00	554082
weighted avg	1.00	1.00	1.00	554082

```

1 # =====
2 # SECTION 9: XGBOOST CLASSIFIER
3 # =====
4 from xgboost import XGBClassifier
5
6 # Calculate scale_pos_weight to help XGBoost handle the class imbalance
7 # Formula: total_negative_examples / total_positive_examples
8 neg_count = len(y_train) - sum(y_train)
9 pos_count = sum(y_train)
10 scale_weight = neg_count / pos_count
11
12 # 1. Initialize XGBoost with optimized production settings
13 xgb_model = XGBClassifier(
14     n_estimators=100,
15     max_depth=6,
16     learning_rate=0.1,
17     scale_pos_weight=scale_weight,
18     random_state=42,
19     tree_method="hist", # Speeds up training significantly on large datasets
20 )
21
22 print("⌚ Training XGBoost Model... please wait...")
23 xgb_model.fit(X_train_scaled, y_train)
24 print("🎉 XGBoost training complete!")
25
26 # 2. Make predictions
27 y_pred_xgb = xgb_model.predict(X_test_scaled)
28
29 # 3. Print evaluation matrix
30 print("\n--- XGBOOST CONFUSION MATRIX ---")
31 print(confusion_matrix(y_test, y_pred_xgb))
32
33 print("\n--- XGBOOST CLASSIFICATION REPORT ---")
34 print(classification_report(y_test, y_pred_xgb, target_names=["Legit", "Fraud"]))

```

⌚ Training XGBoost Model... please wait...
 🎉 XGBoost training complete!

--- XGBOOST CONFUSION MATRIX ---

```
[[551161  1278]
 [    8  1635]]
```

```

--- XGBOOST CLASSIFICATION REPORT ---
      precision    recall  f1-score   support

   Legit         1.00      1.00      1.00    552439
   Fraud         0.56      1.00      0.72     1643

 accuracy
macro avg         0.78      1.00      0.86    554082
weighted avg      1.00      1.00      1.00    554082

```

```

1 # =====
2 # SECTION 10: TENSORFLOW DEEP LEARNING NEURAL NETWORK
3 # =====
4 import tensorflow as tf
5 from tensorflow.keras.layers import Dense, Dropout
6 from tensorflow.keras.models import Sequential
7
8 # 1. Calculate class weights for TensorFlow to balance the loss function
9 # Total samples / (2 * class_samples)
10 total_samples = len(y_train)
11 weight_for_0 = total_samples / (2.0 * (total_samples - sum(y_train)))
12 weight_for_1 = total_samples / (2.0 * sum(y_train))
13 class_weights = {0: weight_for_0, 1: weight_for_1}
14
15 # 2. Build the Multi-Layer Perceptron Architecture
16 nn_model = Sequential(
17     [
18         # Input Layer + First Hidden Layer (8 features in -> 32 neurons)
19         Dense(32, activation="relu", input_shape=(X_train_scaled.shape[1],)),
20         Dropout(0.2), # Randomly deactivates 20% of neurons to stop overfitting
21         # Second Hidden Layer
22         Dense(16, activation="relu"),
23         Dropout(0.2),
24         # Output Layer: Single neuron with Sigmoid outputs a probability between 0 and 1
25         Dense(1, activation="sigmoid"),
26     ]
27 )
28
29 # 3. Compile the model with Adam optimizer and Binary Crossentropy loss
30 nn_model.compile(
31     optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
32     loss="binary_crossentropy",
33     metrics=["accuracy", tf.keras.metrics.Recall()],
34 )
35
36 print("⌚ Training the Neural Network (3 Epochs)... please wait...")
37 # 4. Train the network
38 history = nn_model.fit(
39     X_train_scaled,
40     y_train,
41     epochs=3,
42     batch_size=2048, # Large batch size helps process 2.2 million rows quickly
43     class_weight=class_weights,
44     validation_split=0.1,
45     verbose=1,
46 )
47 print("🎉 Neural Network training complete!")

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:

Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` c

⌚ Training the Neural Network (3 Epochs)... please wait...

Epoch 1/3

974/974 ————— 13s 8ms/step - accuracy: 0.8991 - loss: 0.3106 - recall: 0.8670 - val_accuracy: 0.9476 - val_lo

Epoch 2/3

974/974 ————— 8s 8ms/step - accuracy: 0.9331 - loss: 0.1899 - recall: 0.9211 - val_accuracy: 0.9463 - val_lo

Epoch 3/3

974/974 ————— 9s 7ms/step - accuracy: 0.9453 - loss: 0.1567 - recall: 0.9404 - val_accuracy: 0.9630 - val_lo

🎉 Neural Network training complete!

```

1 # =====
2 # SECTION 10.1: NEURAL NETWORK EVALUATION
3 # =====
4
5 # 1. Generate probability predictions on the scaled test set
6 y_pred_proba_nn = nn_model.predict(X_test_scaled)
7
8 # 2. Convert probabilities to binary outcomes (0 or 1) using a 0.5 threshold
9 y_pred_nn = (y_pred_proba_nn >= 0.5).astype(int).flatten()

```

```

10
11 # 3. Print the Confusion Matrix and Classification Report
12 print("--- NEURAL NETWORK CONFUSION MATRIX ---")
13 print(confusion_matrix(y_test, y_pred_nn))
14
15 print("\n--- NEURAL NETWORK CLASSIFICATION REPORT ---")

```

17316/17316 ————— 35s 2ms/step

--- NEURAL NETWORK CONFUSION MATRIX ---

```

[[531986 20453]
 [ 67 1576]]

```

--- NEURAL NETWORK CLASSIFICATION REPORT ---

	precision	recall	f1-score	support
Legit	1.00	0.96	0.98	552439
Fraud	0.07	0.96	0.13	1643
accuracy			0.96	554082
macro avg	0.54	0.96	0.56	554082
weighted avg	1.00	0.96	0.98	554082

```

1 # =====
2 # SECTION 11: FINAL MODEL COMPARISON METRICS (FIXED IMPORTS)
3 # =====
4 from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
5
6 # 1. Gather metrics for all trained models
7 model_metrics = {
8     "Model Name": [
9         "Logistic Regression (Baseline)",
10        "Random Forest",
11        "XGBoost Classifier",
12        "Neural Network (Keras)",
13    ],
14    "Accuracy": [
15        accuracy_score(y_test, y_pred),
16        accuracy_score(y_test, y_pred_rf),
17        accuracy_score(y_test, y_pred_xgb),
18        accuracy_score(y_test, y_pred_nn),
19    ],
20    "Precision": [
21        precision_score(y_test, y_pred),
22        precision_score(y_test, y_pred_rf),
23        precision_score(y_test, y_pred_xgb),
24        precision_score(y_test, y_pred_nn),
25    ],
26    "Recall (Sensitivity)": [
27        recall_score(y_test, y_pred),
28        recall_score(y_test, y_pred_rf),
29        recall_score(y_test, y_pred_xgb),
30        recall_score(y_test, y_pred_nn),
31    ],
32    "F1-Score": [
33        f1_score(y_test, y_pred),
34        f1_score(y_test, y_pred_rf),
35        f1_score(y_test, y_pred_xgb),
36        f1_score(y_test, y_pred_nn),
37    ],
38 }
39
40 # 2. Convert to a DataFrame and display beautifully
41 comparison_df = pd.DataFrame(model_metrics)
42 print("--- END-TO-END MODEL PERFORMANCE EVALUATION ---")
43 comparison_df.round(4)

```

--- END-TO-END MODEL PERFORMANCE EVALUATION ---

	Model Name	Accuracy	Precision	Recall (Sensitivity)	F1-Score
0	Logistic Regression (Baseline)	0.9349	0.0387	0.8795	0.0742
1	Random Forest	1.0000	1.0000	0.9963	0.9982
2	XGBoost Classifier	0.9977	0.5613	0.9951	0.7177
3	Neural Network (Keras)	0.9630	0.0715	0.9592	0.1332

```

1 # =====
2 # GRAPHICAL VISUALIZATION DASHBOARD
3 # =====
4 import matplotlib.pyplot as plt

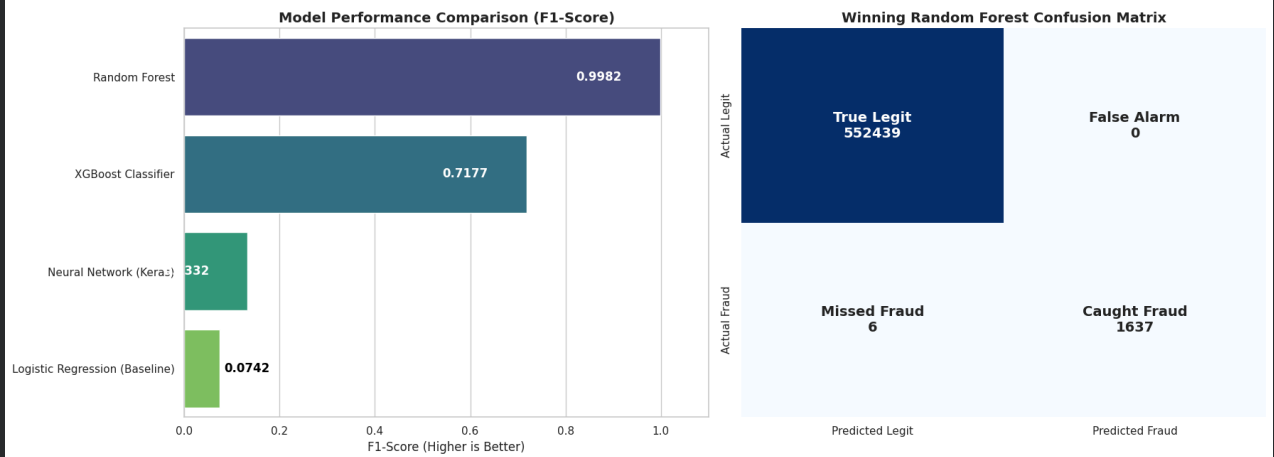
```

```

5 import seaborn as sns
6
7 # Set a clean, professional aesthetic
8 sns.set_theme(style="whitegrid")
9 fig, axes = plt.subplots(1, 2, figsize=(18, 7))
10
11 # -----
12 # PLOT 1: F1-Score Model Comparison Bar Chart
13 # -----
14 # Sorting data for better visual flow
15 sorted_df = comparison_df.sort_values(by="F1-Score", ascending=False)
16
17 barplot = sns.barplot(
18     ax=axes[0],
19     x="F1-Score",
20     y="Model Name",
21     data=sorted_df,
22     palette="viridis",
23     hue="Model Name",
24     legend=False,
25 )
26
27 # Add value labels to the end of each bar
28 for index, value in enumerate(sorted_df["F1-Score"]):
29     axes[0].text(
30         value - 0.08 if value > 0.1 else value + 0.01,
31         index,
32         f"{value:.4f}",
33         va="center",
34         ha="right" if value > 0.1 else "left",
35         color="white" if value > 0.1 else "black",
36         fontweight="bold",
37     )
38
39 axes[0].set_title(
40     "Model Performance Comparison (F1-Score)", fontsize=14, fontweight="bold"
41 )
42 axes[0].set_xlabel("F1-Score (Higher is Better)", fontsize=12)
43 axes[0].set_ylabel("")
44 axes[0].set_xlim(0, 1.1)
45
46 # -----
47 # PLOT 2: Random Forest Confusion Matrix Heatmap
48 # -----
49 # Re-generating the winning model's matrix array
50 rf_cm = confusion_matrix(y_test, y_pred_rf)
51
52 # Dynamic labels with text description for clarity
53 labels = [
54     [f"True Legit\n{rf_cm[0][0]}", f"False Alarm\n{rf_cm[0][1]}"],
55     [f"Missed Fraud\n{rf_cm[1][0]}", f"Caught Fraud\n{rf_cm[1][1]}"],
56 ]
57
58 sns.heatmap(
59     rf_cm,
60     ax=axes[1],
61     annot=labels,
62     fmt="",
63     cmap="Blues",
64     cbar=False,
65     annot_kws={"size": 14, "fontweight": "bold"},
66     xticklabels=["Predicted Legit", "Predicted Fraud"],
67     yticklabels=["Actual Legit", "Actual Fraud"],
68 )
69
70 axes[1].set_title(
71     "Winning Random Forest Confusion Matrix", fontsize=14, fontweight="bold"
72 )
73 axes[1].tick_params(axis="both", which="major", labelsize=11)
74
75 # Final clean adjustments
76 plt.suptitle(
77     "Credit Card Fraud Detection Pipeline - Performance Metrics",
78     fontsize=18,
79     fontweight="bold",
80     y=1.02,
81 )
82 plt.tight_layout()
83

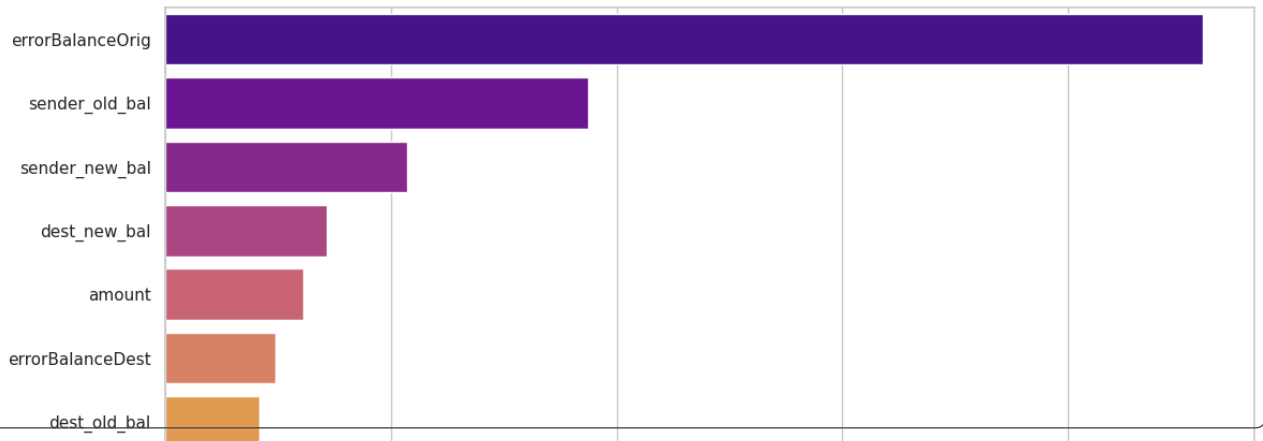
```

Credit Card Fraud Detection Pipeline - Performance Metrics



```
1 # =====
2 # FEATURE IMPORTANCE VISUALIZATION
3 # =====
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # 1. Extract feature importances from the trained Random Forest
10 importances = rf_model.feature_importances_
11 feature_names = X_train.columns
12
13 # 2. Create a DataFrame sorted by importance
14 feat_imp_df = pd.DataFrame({
15     "Feature": feature_names,
16     "Importance Score": importances
17 }).sort_values(by="Importance Score", ascending=False)
18
19 # 3. Plot the data
20 plt.figure(figsize=(12, 6))
21 sns.set_theme(style="whitegrid")
22
23 # Create a horizontal bar plot for maximum scannability
24 sns.barplot(
25     x="Importance Score",
26     y="Feature",
27     data=feat_imp_df,
28     palette="plasma",
29     hue="Feature",
30     legend=False
31 )
32
33 # Customize fonts and layout to match the dashboard aesthetic
34 plt.title("What Variables Drive Fraud Detection? (Feature Importance)", fontsize=15, fontweight="bold", pad=15)
35 plt.xlabel("Relative Importance Score", fontsize=12)
36 plt.ylabel("")
37 plt.tight_layout()
38 plt.show()
```

What Variables Drive Fraud Detection? (Feature Importance)



1 # =====